UTAA - Faculty of Engineering

CENG 454: Game Programming

Spring Term 2025-2026

Project: Sky-High Prototype II

Threat Corridor — Take-Home Midterm Report

Full Name	Egemen Caliskan
Student ID	220444035
GitHub URL	https://github.com/Jeithx/CENG454-HW2-Egemen-Caliskan-220444035
Submission Date	08 April 2026

Section 1: Screenshots

Screenshot 1: HW1 Baseline Import in HW2 Repository

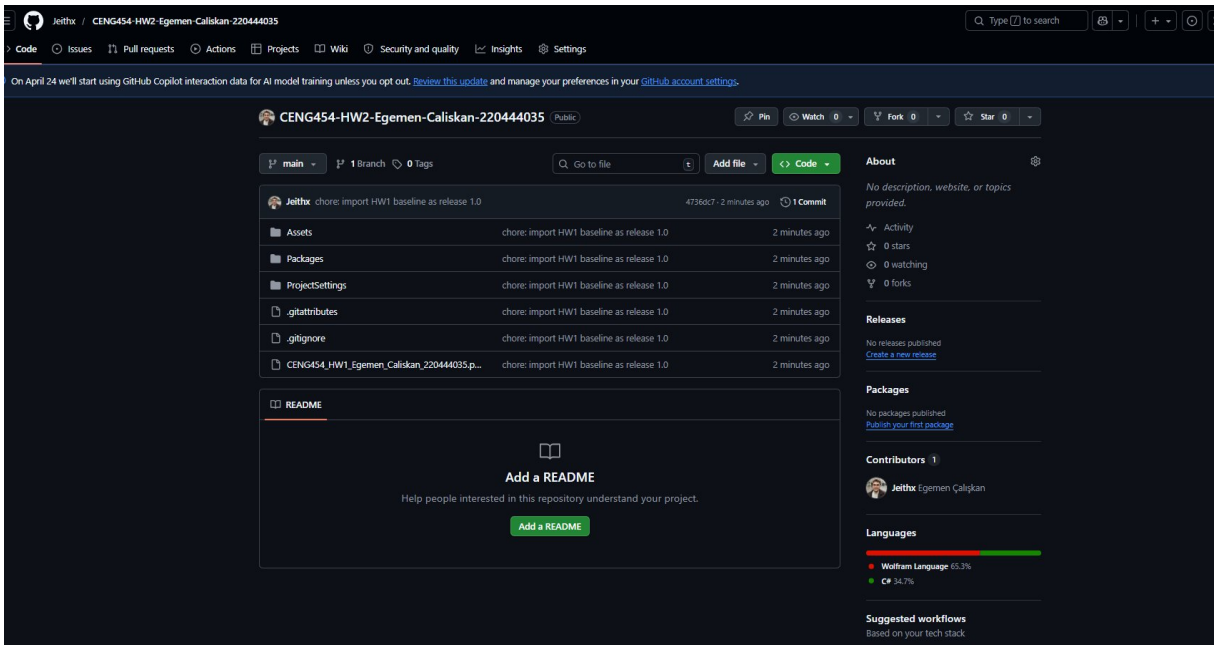

Figure 1: HW1 baseline successfully imported into the new HW2 repository on GitHub

Screenshot 2: Scene View - Terrain, Runway, Landing Strip, and Danger Zone

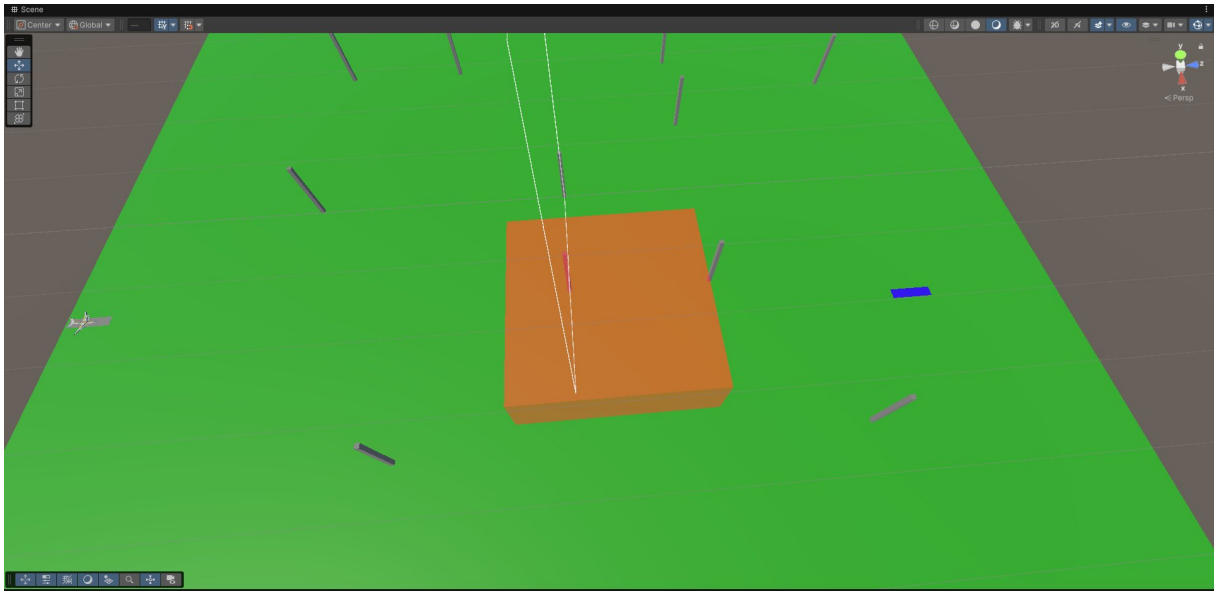

Figure 2: Bird's eye Scene View showing terrain, takeoff runway, landing strip, danger zone volume, and building obstacles

Screenshot 3: DangerZoneController Inspector

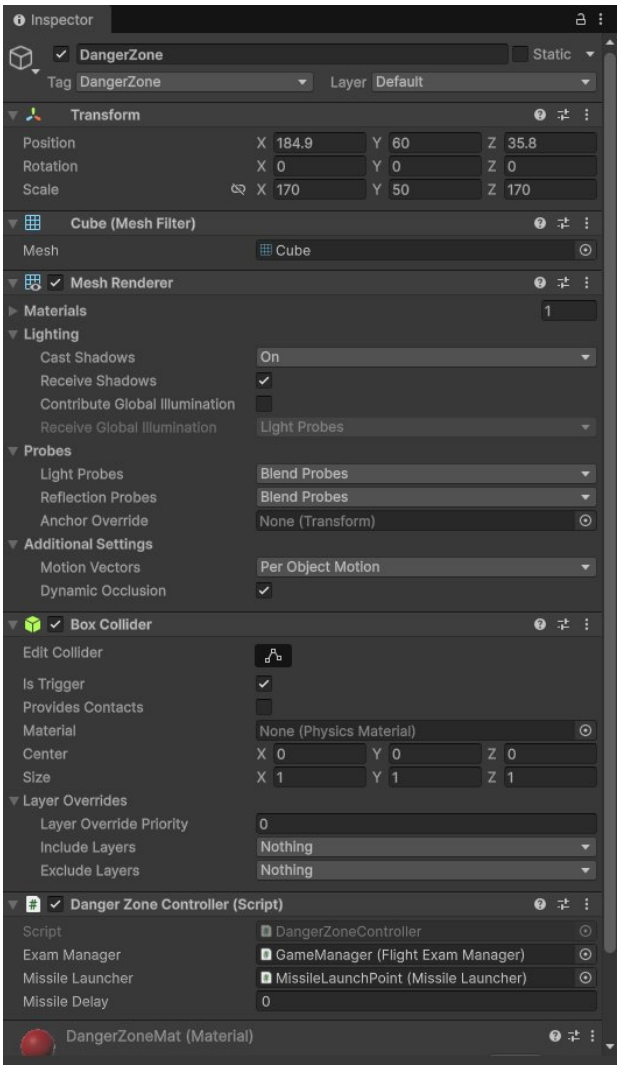

Figure 3: DangerZone Inspector showing Box Collider (Is Trigger), DangerZoneController with Exam Manager and Missile Launcher references

Screenshot 4: Missile Prefab Inspector

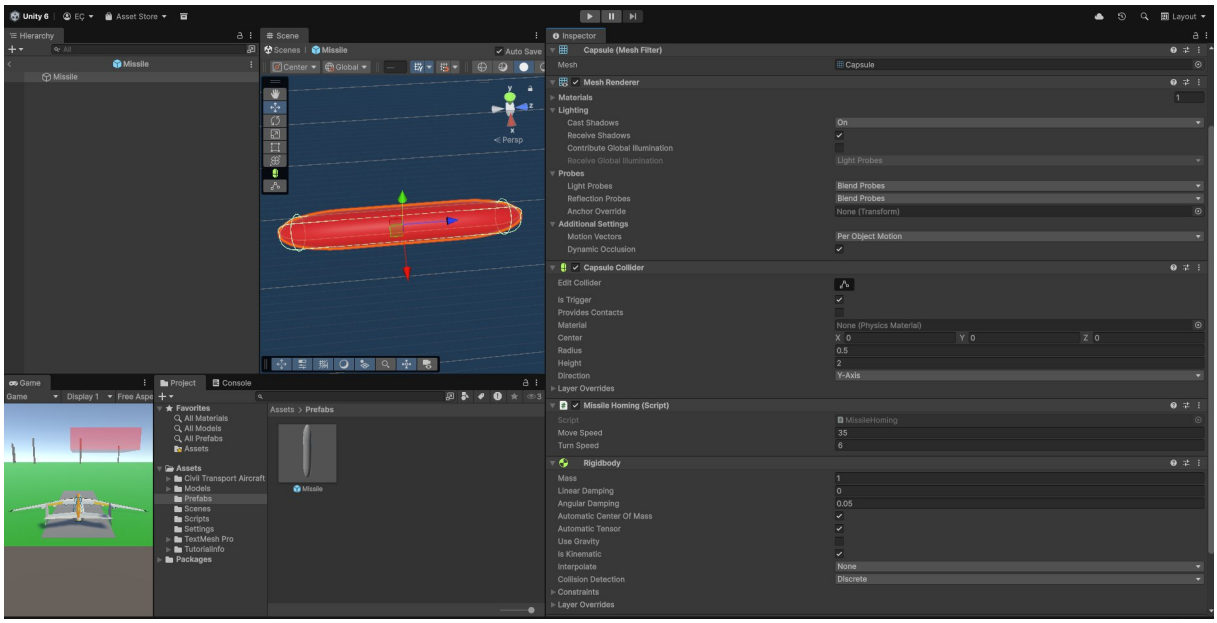

Figure 4: Missile prefab Inspector showing MissileHoming (moveSpeed=35, turnSpeed=6), Rigidbody (no gravity, kinematic), and Capsule Collider (Is Trigger)

Screenshot 5: HUD Warning - "Entered a Dangerous Zone!"

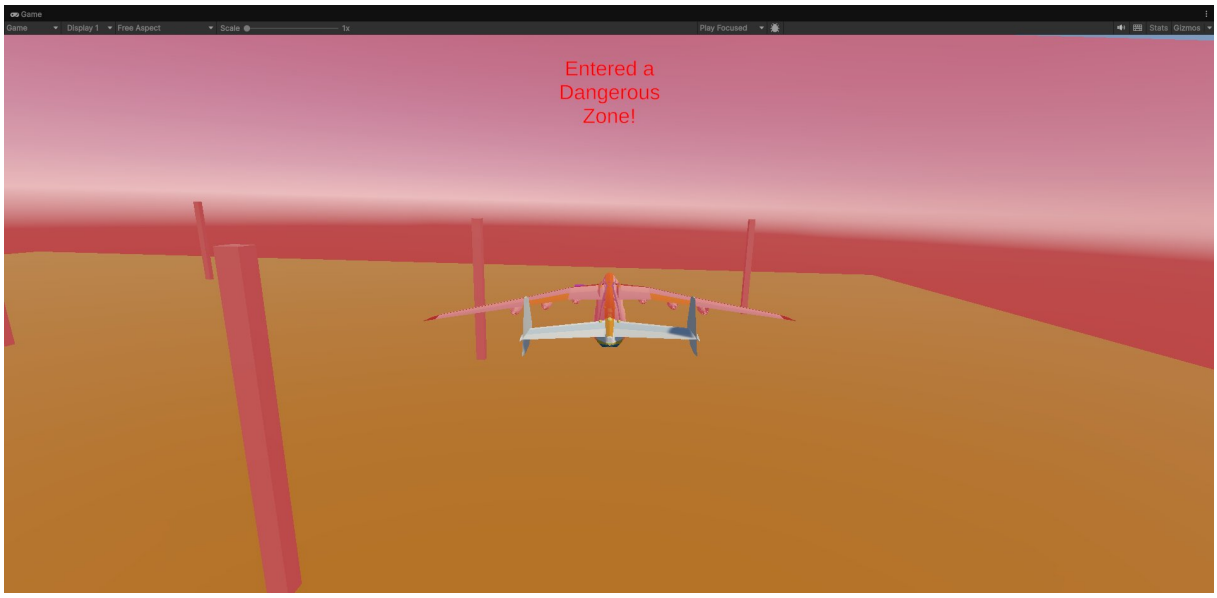

Figure 5: Game View showing "Entered a Dangerous Zone!" HUD warning upon entering the danger zone

Screenshot 6: Missile Pursuing the Aircraft

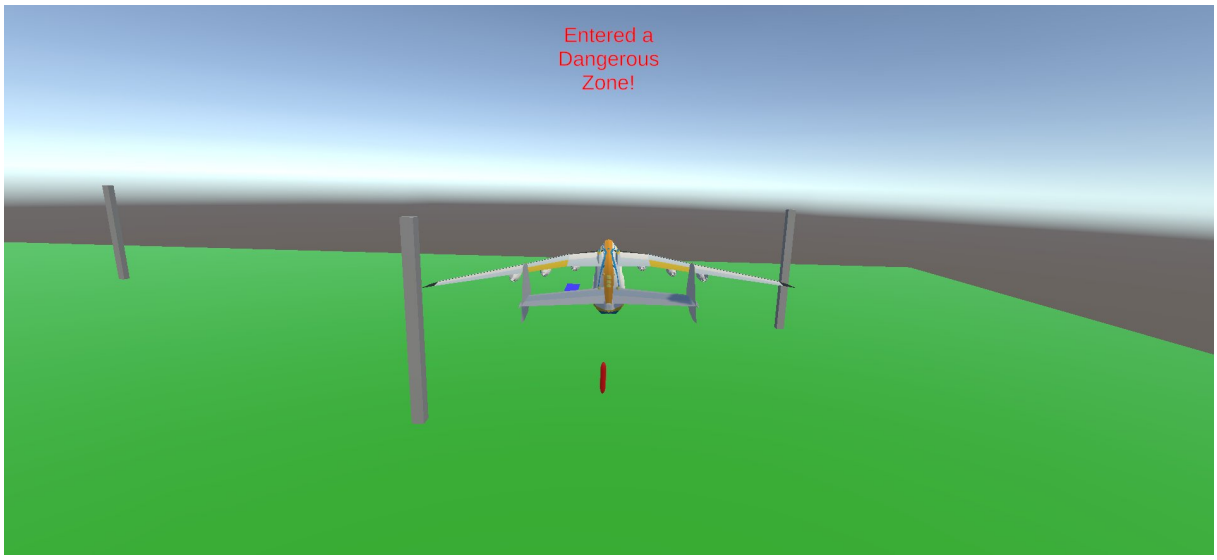

Figure 6: Game View showing the missile launched from the ground and pursuing the aircraft inside the danger zone

Screenshot 7: Mission Complete After Safe Landing

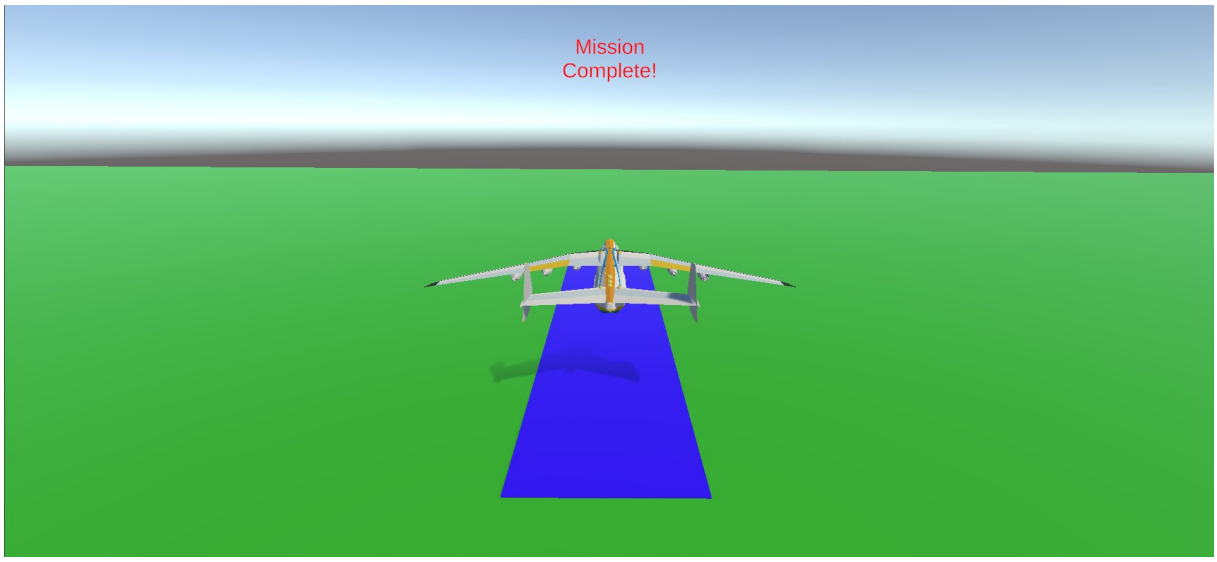

Figure 7: Game View showing "Mission Complete!" after the aircraft landed safely on the landing strip following threat clearance

Section 2: Code Analysis

(a) How I Split Responsibility Between Systems

While you are building something in such an object oriented environment like Unity, you can't just put everything in a single god class. Zone detection, missile spawning, HUD text, landing check, shouldn't be all in one place. Your project will get messy real fast that you don't even know how. You would be dealing with a lot of lines in one Update() and couldn't tell what was doing what after a while. You should split it up.

In my project, DangerZoneController's only job is to notice when the player enters or leaves, then tell other scripts about it. It doesn't know anything about missiles or landing. MissileLauncher handles instantiating the missile prefab and playing the launch sound. MissileHoming goes on the missile prefab and just handles how will the missile fly towards to target and it's math every frame. AircraftThreatHandler is located on the plane and deals with what happens when a missile actually hits you. And FlightExamManager is like the referee — it keeps track of where you are in the mission (took off? entered zone? threat cleared? landed?) and updates the HUD text.

The nice thing about splitting it this way is I could test each piece alone. Missile not tracking? I only need to look at MissileHoming. HUD not updating? That's FlightExamManager. If everything was in one script, a bug in the landing check would somehow break the missile spawning and I'd spend forever figuring out why.

(b) How I Tuned the Missile and Danger Zone

This took a lot of trial and error, not gonna lie. First version of the missile was way too fast — the thing would lock on and hit you before you could even turn. That's not fun, that's just unfair. Then I made it too slow and you could literally fly in a straight line and it would never catch up. Also not great.

I ended up setting moveSpeed to around 35 and turnSpeed to about 6. That means the missile is fast enough that you feel the pressure, but its turning radius and speed is enough that if you keep going forward out of the danger zone, you can easily escape it. The danger zone box I made is 170x50x170 units so you can feel the pressure of the missile but you can also see the edges and know where to escape to. I also put the launch point on the ground kind of near the center of the zone so you can actually see the missile coming up at you.

(c) Audio and Cleanup Between States

I used three AudioSource components for the key moments. One of them plays a warning siren when you enter the zone, one plays a launch whoosh when the missile fires, and one plays either an explosion (if the missile hits you) or a radio chime (if you land safely). The tricky part was making sure sounds don't bleed into wrong states. Like, the warning siren should stop when you leave the zone. If I forgot to stop it, you'd be flying over peaceful terrain with an alarm still blaring.

For cleanup, everything goes through the OnTriggerExit path on DangerZoneController. When the player leaves the zone: stop the warning sound, cancel the coroutine countdown if it's still going, and if a missile exists, call Destroy on it. I also set the activeMissile reference to null after destroying it because I learned the hard way that a destroyed GameObject isn't actually null right away in Unity — it's in this weird limbo state for a frame. If you try to access it you get a MissingReferenceException. Setting it to null explicitly avoids that.

Section 3: Analysis Questions

Why is a trigger volume a good fit for the danger zone?

A trigger volume lets you detect when something enters or exits a region without any physical collision happening. That's exactly what we need here — the plane should fly through the zone freely, not bounce off an invisible wall. Regular colliders would block the aircraft or send it spinning, which makes no sense for an airspace boundary. With a trigger (Is Trigger checked on the BoxCollider), OnTriggerEnter and OnTriggerExit fire but nothing physically pushes the plane. It's like an invisible sensor that says "hey, the player is in this area now" without interfering with the flight.

Why must the systems coordinate through shared mission state?

Because things depend on each other and the order matters. You can't land successfully if you haven't cleared the threat yet. The missile shouldn't spawn if you already left the zone. The HUD needs to know whether you're in danger or safe. If each script just did its own thing without checking the shared state, you'd get stuff like: the landing pad says "mission complete" even though you never entered the danger zone, or the missile spawns twice because two scripts both thought they were responsible for launching it. FlightExamManager is the single source of truth. Everything checks in with it before making decisions. That way there's one place to look when something goes wrong.

Why must the countdown be canceled on early zone exit?

Because if you don't cancel it, you get a ghost missile. The coroutine is already running — it was started when you entered the zone. If you fly out after 2 seconds, the coroutine doesn't care. It's still counting. Three seconds later it fires off a missile aimed at a player who's no longer in the zone. That missile has no reason to exist. Worse, the DangerZoneController might have already cleaned up its references, so now you have an orphan missile flying around with nobody managing it. StopCoroutine on the cached Coroutine reference is the fix. You call it in OnTriggerExit, the countdown dies, no missile spawns. Clean.

How did you choose missile speed and tracking so the encounter is readable?

I wanted the player to see the missile, understand what it's doing, and have a window to react. If the missile is too fast, you die before you even know it launched. If the turn rate is too high, it just glues to your position and there's no outrunning it. I set moveSpeed at 35 and turnSpeed at 6. What this means in practice: the missile covers ground quickly but it can't make sharp turns. If you don't lose speed or keep banking the plane, you won't get hit. Missile will give enough time you time to reach the zone edge. It took maybe 15-20 test runs to land on those numbers. I kept adjusting by 1 or 2 at a time until it felt right.

Why separate zone detection, launching, homing, and hit response?

Single responsibility, basically. Each script does one thing. If I shoved everything into one aircraft script, I'd have OnTriggerEnter checking if it's a zone or a missile, Update doing homing math AND mission state AND HUD updates, and the whole thing would be a tangled mess. Debugging becomes a nightmare because changing one thing might break three others.

With separate scripts, if the missile isn't turning right, I open MissileHoming.cs and that's the only file I touch. If the HUD is showing the wrong message, that's FlightExamManager. If the missile spawns at the wrong time, that's DangerZoneController's coroutine logic. Each bug has a home address. Also, if I wanted to add a second danger zone with a different launcher setup, I just drop another DangerZoneController on a new trigger volume and wire up a different MissileLauncher. No code changes needed. That wouldn't be possible if everything was hardcoded into one monolithic script on the aircraft.

What would you build next with one more sprint?

Probably a radar or minimap that shows you where the missile is relative to your aircraft. Right now you have to physically look behind you (or move the camera) to see if the missile is gaining on you, which is hard when you're also trying to navigate toward the zone edge. A small UI element in the corner showing a blip for the missile direction would make the gameplay a lot more readable without making it easier. You'd still have to dodge, but at least you'd know which direction to dodge. I think that's more useful than adding extra missiles or health bars, because the core problem right now is awareness, not difficulty.

Section 4: Sprint Retrospective

The hardest part of this midterm wasn't any single system, it was making them all talk to each other without stepping on each other's toes. Getting the missile to launch was easy. Getting it to stop when it should, clean up after itself, not leave ghost references behind — that was the real challenge. I spent a solid chunk of time on a bug where the missile would spawn even after I left the zone. Turned out I was starting the coroutine but never caching the reference, so StopCoroutine had nothing to stop. Once I stored the Coroutine return value and used that for cancellation, the ghost missile issue went away.

The terrain stuff was actually kind of fun. I've never used Unity's terrain tools before and it was cool to set up the environment. Probably spent too long on that compared to how many points it's worth, but whatever. The landing validation was simpler than I expected — just a boolean check on threatCleared before accepting the OnTriggerEnter from the landing zone.

If I could redo this, I'd write the FlightExamManager first with all the state booleans and HUD hooks before touching any other script. I ended up retrofitting the mission state tracking after I'd already written the zone and missile scripts, and that meant rewiring a bunch of references that should have been there from the start. Start with the manager, build the systems around it. That's the lesson.

Section 5: Troubleshooting — Midterm Report #003

The Ghost Missile

(a) Why the missile still appears after exit

The student started a coroutine in OnTriggerEnter that waits 5 seconds and then spawns a missile. The pilot leaves after 2 seconds, OnTriggerExit fires, the warning text gets cleared, everything looks fine. But we missed telling the coroutine to stop. It's still running in the background, counting down. Three seconds later it hits zero and calls the launch method. So now you've got a missile flying around the scene going after a player who already left. Nobody is keeping track of it either the DangerZoneController already did its exit cleanup and moved on. That missile is basically is not connected to any parent.

(b) Which timer, coroutine, or state cleanup was forgotten

The student probably forgot to save the coroutine into a variable when they called StartCoroutine. Or maybe they did save it, but then didn't bother calling StopCoroutine inside OnTriggerExit. Either way the result is the same — the countdown keeps going even after the player is gone. Coroutines don't just stop on their own when trigger conditions change. They keep running until they finish or until you manually kill them. While starting a coroutine is relatively easy to handle and remember, making it stop can be easily forgettable.

(c) The exact logic needed on zone exit to prevent ghost spawns

OnTriggerExit needs to handle three things, in this order. Firstly if there's a countdown coroutine still running, we should stop it with StopCoroutine. Be careful about using the cached Coroutine reference for this, not the string method name because the string version doesn't always work right. Secondly, check if a missile already exists in the scene. Maybe the full 5 seconds already passed and the missile is already out there flying. If so, call Destroy on it. And finally, set activeMissile to null after you destroy it. Unity has an annoying issue where a destroyed object isn't technically null for a frame or two. You can still accidentally reference it and get a MissingReferenceException. Setting it to null yourself avoids this error. Do all three of these in OnTriggerExit and you won't have ghost missiles anymore.